

SignBridge

AI Two-Way Sign Language Translator

A line-by-line companion to the single-file HTML application — covering HTML structure, the design-token CSS system, MediaPipe Hands integration, rule-based gesture classification, temporal smoothing, speech I/O, and the offline English translation engine.

MediaPipe Hands
AT A GLANCE

TensorFlow.js

Web Speech API

100% In-Browser

No Backend

2,262

Lines of source

1

Self-contained HTML file

~46

Recognizable gestures

400+

Offline dictionary words

HOW TO READ THIS BOOK

Chapters 1-3 establish the project context, HTML skeleton and visual design system.

Chapters 4-8 walk through the JavaScript core: state, MediaPipe, drawing, classification, temporal smoothing and sign acceptance — the heart of the recognition pipeline.

Chapters 9-11 cover speech I/O, the offline English translation engine, UI wiring and emergency features. Chapter 12 closes with extension points and architectural notes.

Table of Contents

1.1 What is SignBridge?	5
1.2 Technology Stack	5
1.3 Architectural Overview	6
2.1 Document Outline	7
2.2 The Topbar	7
2.3 The Two-Column Main Layout	7
2.4 The Video Panel	8
2.5 The English Translation Panel	8
3.1 Design Tokens	9
3.2 High-Contrast Override	9
3.3 Typography	10
3.4 Signature Animations	10
3.5 Confidence Meter Styling	10
4.1 Tuning Constants	11
4.2 Gesture Labels and Word Dictionary	11
4.3 Mutable Application State	11
4.4 DOM Reference Cache	12
5.1 Initialising MediaPipe	13
5.2 Starting the Camera	13
5.3 The Detection Loop	14
5.4 FPS Counter and Session Clock	14
6.1 The onHandResults Entry Point	15
6.2 The 21 Landmarks and Their Connections	16

6.3 Drawing Connections and Joints	16
7.1 Normalisation	17
7.2 Finger Extension Test	17
7.3 Letter Rules	18
7.4 Numbers and Word-Signs	18
7.5 Geometry Helpers	19
8.1 The Problem: Jittery Classifications	20
8.2 The Sliding-Window Majority Vote	20
8.3 The Hold Counter	21
8.4 The Inter-Letter Delay	21
8.5 Accepting a Sign	21
9.1 Text-to-Speech (TTS)	22
9.2 Speech-to-Text (STT)	22
9.3 Toggling STT	23
10.1 Design Goal: 100% Offline	24
10.2 The Dictionary	24
10.3 Word-Sign Direct Mappings	25
10.4 The Building Buffer	25
10.5 Matching Algorithm	26
10.6 Committing a Word	26
10.7 Rendering the Sentence	26
11.1 Button Wiring	27
11.2 Keyboard Shortcuts	27
11.3 Emergency Mode	28
11.4 History Logging	28

11.5 Toast Notifications and Status Indicators	28
12.1 Architectural Strengths	29
12.2 Known Limitations	29
12.3 Extension Points	29
12.4 Closing Thoughts	30

A note on structure

This book follows the natural reading order of the source file: HTML first, then CSS, then JavaScript from top to bottom. Each chapter lists the relevant line ranges in its opening paragraph so you can keep the source open in a second window while reading.

CHAPTER 1

Introduction & Architecture

What SignBridge is, the technology stack, and how the pieces fit together.

1.1 What is SignBridge?

SignBridge is a single-file HTML application that performs two-way translation between sign language and spoken English, running entirely inside the browser with no backend server. The deaf-or-mute user signs at the webcam; the system recognises each letter or word-sign, builds English text, and optionally speaks it aloud. The hearing user speaks into the microphone; the system transcribes their speech and displays it for the signer to read. Both directions share a single unified interface.

The application is built as one self-contained `index.html` file approximately 2,262 lines long, with all HTML, CSS and JavaScript inlined. The only external dependencies are four CDN-hosted libraries: MediaPipe Hands, MediaPipe Camera Utils, MediaPipe Drawing Utils, and TensorFlow.js (loaded but reserved for a future model upgrade). Because no video or audio ever leaves the browser, SignBridge is suitable for environments where privacy is non-negotiable — schools, hospitals, domestic use, and field deployment on consumer laptops.

1.2 Technology Stack

The stack is intentionally conservative. Each technology was chosen because it ships in every modern browser without installation, while still being powerful enough for real-time perception work. The table below summarises the role of each component.

Layer	Technology	Role in SignBridge
Hand tracking	MediaPipe Hands (WASM)	21-landmark hand model, runs at 25-30 FPS in-browser
Future ML	TensorFlow.js 4.15	Loaded for the planned neural classifier upgrade
Speech out	Web Speech API (SpeechSynthesis)	Reads translated text aloud (TTS)
Speech in	Web Speech API (SpeechRecognition)	Transcribes the hearing user's speech (STT)
Rendering	Canvas 2D API	Draws the skeleton overlay on top of the video feed
Layout	CSS Grid + custom properties	Two-column panel layout, dark neon theme
Logic	Vanilla JavaScript (no framework)	Single <code><script></code> block, ~1,300 lines
Storage	In-memory only	No persistence — page refresh clears all state

1.3 Architectural Overview

SignBridge follows a classic perception-decision-render loop. Each video frame is captured from the webcam, sent through MediaPipe for hand-landmark extraction, classified by a rule-based gesture engine, smoothed across frames by a temporal majority-vote filter, and finally committed to the translation buffer when it has been stable for long enough. The translation buffer feeds two parallel outputs: a raw letter-by-letter panel (the signer's view) and an offline English-translation panel that performs longest-prefix dictionary matching to convert accumulated letters into whole words.

A second, independent path handles speech: the Web Speech API transcribes incoming audio and writes the transcript to a panel that the signer reads. An emergency mode overlays a red banner, attempts geolocation, and speaks an SOS phrase. The two paths never share state except through the shared history log, which keeps the architecture easy to reason about.

Architectural decision: rule-based vs. neural classifier

The classifier is currently rule-based — it tests geometric predicates on the normalised landmark vector. This trades some accuracy for transparency (every decision is auditable), zero cold-start training, and immediate deployability. The code is structured so the function `classifyGesture()` can be replaced with a single `tf.model.predict()` call without touching the rest of the pipeline.

CHAPTER 2

HTML Skeleton & UI Layout

The semantic structure that holds the entire application together.

2.1 Document Outline

The HTML opens with the standard `<!DOCTYPE html>` and a `<head>` block that loads four CDN libraries, imports three Google Fonts (Syne, Inter, JetBrains Mono), and embeds the entire CSS in a single `<style>` block. The `<body>` contains one root `#app` container, an emergency banner, a topbar, a `<main>` element with two columns of panels, a statusbar footer, a toast container, and one large `<script>` block at the end. No external HTML files, no templates, no virtual DOM.

```
<body>
  <div id="emergency-banner">...SOS banner...</div>
  <div id="app">
    <header id="topbar">...logo, language selector, control buttons...</header>
    <main id="main">
      <div class="panel">...video + sign detection...</div>
      <div class="right-col">
        <div class="panel">...Sign to Text...</div>
        <div class="panel">...Speech to Text...</div>
        <div class="panel">...History...</div>
      </div>
      <div class="panel" id="eng-translation-panel">...English Translation...</div>
    </main>
    <footer id="statusbar">...camera, MediaPipe, speech, time...</footer>
  </div>
  <div id="toast"></div>
  <script>...all JavaScript...</script>
</body>
```

Listing 2.1 — High-level body outline (line ~724).

2.2 The Topbar

The topbar hosts the SignBridge logo (with a small "AI Beta" badge), a language selector with four sign-language options (ASL, PSL, BSL, ISL), and five icon buttons: mute toggle, high-contrast toggle, fullscreen, emergency SOS, and (within the detection panel) record. Each icon button shares the `.icon-btn` class so they all get the same 36 by 36 pixel hit area, hover state, and focus ring for keyboard accessibility.

2.3 The Two-Column Main Layout

The `#main` element is a CSS grid with two equal columns. The left column holds the video panel — by far the visually largest component — and the right column holds three stacked panels: Sign-to-Text (the signer's output), Speech-to-Text (the hearing user's input), and Conversation History. A fourth full-width panel below the grid, `#eng-translation-panel`, hosts the offline English translation engine. On viewports narrower than 900 pixels a media query collapses the grid into a single column for mobile usability.

2.4 The Video Panel

Inside `#video-container` there are six stacked elements, all absolutely positioned to fill the container: the `<video>` element showing the webcam feed (mirrored with `scaleX(-1)` for natural interaction), a `<canvas>` overlay for drawing landmarks, an animated scanner line that sweeps top-to-bottom every 3 seconds, four corner brackets (the classic targeting reticle), a stats pill row showing FPS and hand count, and a sign overlay badge showing the currently detected letter and its confidence percentage. Before the camera starts, a `#cam-overlay` covers everything with a "Start Camera" call-to-action.

Why mirror the video?

Selfie-style mirroring (`transform: scaleX(-1)`) makes the user feel like they are looking into a mirror — when they raise their right hand, the on-screen hand also rises on the right side. The landmark canvas is mirrored identically so the drawn skeleton stays aligned with the video. The mirroring is purely visual; MediaPipe still processes the un-mirrored frame, so the `handedness` label reports the anatomically correct hand.

2.5 The English Translation Panel

The full-width `#eng-translation-panel` at the bottom is the most information-dense region of the UI. From top to bottom it contains: a header with a "100% OFFLINE" badge and sign-count plus three action buttons (clear, copy, speak); a row of sign chips showing every accepted sign in order; a "Building:" line that shows the current letter buffer with a blinking cursor and a live word-match indicator; the main output area showing the committed English sentence in large type; a row of word suggestions derived from the current buffer; and a meanings strip that automatically lists definitions for every word in the sentence, drawn from the offline dictionary.

CHAPTER 3

CSS Design System

Design tokens, neon palette, and the visual identity of SignBridge.

3.1 Design Tokens

All visual properties are defined once in the `:root` custom-property block and consumed everywhere else. This makes the entire theme tweakable from a single place — change `--cyan` and every accent, glow, and chip updates with it. The tokens are grouped by purpose: core palette, neon accents, text colours, glow helpers, and layout radii.

```
@import url('https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800'
&family=Inter:wght@300;400;500;600&family=JetBrains+Mono:wght@400;500'
&display=swap');

:root {
  /* Core palette - deep void navy through raised surface */
  --void: #07080f;
  --deep: #0d0f1e;
  --surface: #131627;
  --raised: #1a1e35;
  --border: #252a47;
  --muted: #3a4060;

  /* Neon accents */
  --cyan: #00e5ff;
  --cyan-dk: #007a8c;
  --green: #39ff6a;
  --amber: #ffbe00;
  --rose: #ff4d6d;

  /* Text */
  --txt-hi: #eef0ff;
  --txt-md: #9ba3c8;
  --txt-lo: #555e85;

  /* Glow helpers - reused on every neon element */
  --glow-cyan: 0 0 8px #00e5ff66, 0 0 24px #00e5ff33;
  --glow-green: 0 0 8px #39ff6a55, 0 0 20px #39ff6a22;
  --glow-amber: 0 0 8px #ffbe0066, 0 0 20px #ffbe0033;

  /* Layout radii */
  --r-sm: 8px; --r-md: 14px; --r-lg: 20px;
}
```

Listing 3.1 — The design-token block (lines 28-58).

3.2 High-Contrast Override

Accessibility is built in from the start. A single `body.hc` class redefines every token to a high-contrast alternative: pure black backgrounds, brighter neons, and pure-white text. Because every component reads from the same tokens, the high-contrast mode is essentially free — one class toggle on the body, no component-level overrides needed. The H keyboard shortcut flips this state instantly.

3.3 Typography

Three font families are imported, each with a specific role. Syne — a wide, technical display face — is used for the logo, panel titles, sign overlay letter, and any element that needs to feel like instrumentation. Inter handles all body copy and button labels. JetBrains Mono is reserved for technical readouts: FPS counters, confidence percentages, history timestamps, and the badge tags. The visual rhythm of the interface comes largely from this three-way contrast.

3.4 Signature Animations

Three animations define the visual personality of SignBridge. The `scan` animation moves the cyan scanner line top-to-bottom across the video feed every 3 seconds, giving the impression of continuous active sensing. The `pulse` animation fades the green status dots in the panel headers between full and 40% opacity, creating a "live" heartbeat. The `mic-pulse` animation expands a rose-coloured ring around the microphone button while recording, providing immediate feedback that audio capture is active.

```
/* The signature scanner line - sweeps every 3 seconds */
#scanner-line {
  position: absolute; left: 0; right: 0; height: 2px;
  background: linear-gradient(90deg, transparent, var(--cyan), transparent);
  box-shadow: 0 0 10px var(--cyan), 0 0 30px var(--cyan-dk);
  top: -2px;
  animation: scan 3s linear infinite;
}
@keyframes scan {
  0% { top: -2px; opacity: 0; }
  5% { opacity: 1; }
  95% { opacity: 1; }
  100% { top: 100%; opacity: 0; }
}
```

Listing 3.2 — The scanner-line signature animation (lines 227-241).

3.5 Confidence Meter Styling

The confidence bar at the foot of the video panel uses three CSS classes — `.low`, `.mid`, `.high` — to switch its fill colour from rose to amber to cyan as confidence rises. Each class also changes the glow shadow, so the bar visually intensifies as detection becomes more certain. The same pattern is used for the stability bar, but with a fixed green fill because stability is intrinsically a "good when high" metric.

CHAPTER 4

JavaScript State & Constants

The configuration knobs and mutable state that drive the application.

4.1 Tuning Constants

The first thing the JavaScript block does is declare four tuning constants that govern the recognition pipeline. These are the levers you would adjust if you wanted SignBridge to behave more aggressively (accept signs faster, lower thresholds) or more conservatively (require longer holds, higher confidence).

```
const CONFIDENCE_THRESHOLD = 0.72; // minimum confidence to accept a sign
const HOLD_FRAMES          = 18;   // frames a sign must remain stable
const LETTER_DELAY_MS      = 700;  // min ms between accepted letters
const MAX_CONF_HISTORY     = 60;   // graph data points to retain
```

Listing 4.1 — Tuning constants (lines 952-955).

The 0.72 confidence threshold balances false positives against responsiveness. The 18-frame hold requirement (about 0.6 seconds at 30 FPS) prevents brief misclassifications from polluting the buffer; a sign must be detected consistently for more than half a second before it is committed. The 700 ms inter-letter delay stops the same held sign from being accepted repeatedly, which would otherwise flood the translation buffer with duplicate letters.

4.2 Gesture Labels and Word Dictionary

The `GESTURE_LABELS` array enumerates every pattern the classifier can return: the 26 letters A-Z, the digits 0-9, and ten common word-signs (HELLO, THANK YOU, YES, NO, PLEASE, SORRY, HELP, WATER, FOOD, EMERGENCY). The `WORD_DICT` array is a smaller list of about 30 common words used by the autocomplete suggestion engine in the Sign-to-Text panel.

4.3 Mutable Application State

All runtime state is held in plain top-level `let` variables — there is no state container, reducer, or observable system. This is appropriate for an application of this size: the state surface is small enough that direct mutation is readable, and the absence of a framework keeps the bundle trivially small.

Variable	Type	Purpose
<code>cameraRunning</code>	boolean	Whether the webcam stream is active
<code>handsReady</code>	boolean	Whether the MediaPipe model has finished loading
<code>muted</code>	boolean	Whether TTS output is suppressed
<code>highContrast</code>	boolean	Whether the HC accessibility theme is on
<code>recording</code>	boolean	Whether gesture dataset recording is active
<code>lastLetter</code>	string	The most recently accepted sign label

lastLetterTime	number	Timestamp of the last acceptance (ms)
holdCounter	number	Frames the current candidate has been stable
holdCandidate	string	The candidate sign currently being held
completedText	string	Committed words in the Sign-to-Text panel
currentBuilding	string	Letters being accumulated into a word
confHistory	number[]	Confidence values for the live graph
historyEntries	object[]	Conversation history items
emergencyActive	boolean	Whether SOS mode is currently active

Table 4.1 — The core mutable state variables (lines 977-995).

4.4 DOM Reference Cache

After the state block, every DOM element the script will touch is fetched once with `getElementById` and stored in a `const`. This avoids repeated lookups in the hot detection loop — a micro-optimisation that matters when `onHandResults` runs 30 times per second. The helper `const $ = id => document.getElementById(id);` gives the cache declarations a jQuery-like terseness without pulling in any library.

CHAPTER 5

MediaPipe Hands & Camera Pipeline

How frames flow from webcam to landmark data, frame after frame.

5.1 Initialising MediaPipe

The `initMediaPipe()` function constructs a new `Hands` instance configured to detect up to two hands at model complexity 1 (the middle of three quality levels), with a detection confidence of 0.7 and a tracking confidence of 0.6. The `locateFile` callback tells MediaPipe where to fetch its WASM and model assets — the same CDN that serves the JavaScript. After construction, `hands.initialize()` is awaited; only once that promise resolves is `handsReady` flipped to true, which the detection loop checks before sending any frames.

```
function initMediaPipe() {
  hands = new Hands({
    locateFile: file =>
      `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
  });
  hands.setOptions({
    maxNumHands: 2,
    modelComplexity: 1,
    minDetectionConfidence: 0.7,
    minTrackingConfidence: 0.6
  });
  hands.onResults(onHandResults);
  hands.initialize().then(() => {
    handsReady = true;
    updateStatus('st-mp', 'st-mp-lbl', true, 'MediaPipe ready');
    toast('Hand detection engine ready');
  }).catch(err => {
    console.error('MediaPipe init error:', err);
  });
}
```

Listing 5.1 — MediaPipe initialisation (lines 1021-1047).

5.2 Starting the Camera

`startCamera()` requests the user's webcam via `navigator.mediaDevices.getUserMedia()`, asking for a 640 by 480 user-facing stream with no audio. The stream is attached to the `<video>` element, and once metadata has loaded the overlay is hidden, the `cameraRunning` flag is set, and three subsystems are started: the detection loop, the FPS counter, and the session clock. If the user denies camera permission a toast message is shown and the error is logged — the application fails gracefully rather than throwing.

5.3 The Detection Loop

The detection loop is an async function that reschedules itself with `requestAnimationFrame`. On every iteration, if `MediaPipe` is ready and the video has at least two frames of buffered data, it sends the current video frame to `hands.send({ image: webcamEl })`. This call is asynchronous and resolves once `MediaPipe` has finished processing — at which point `onHandResults` has already been invoked with the results. The canvas size is also synchronised with the video dimensions on each frame to handle dynamic resolution changes.

```
function startDetectionLoop() {
  async function loop() {
    if (!cameraRunning) return;
    if (handsReady && webcamEl.readyState >= 2) {
      try {
        await hands.send({ image: webcamEl });
      } catch(e) { /* ignore frame errors */ }
    }
    if (canvasEl.width !== webcamEl.videoWidth) {
      canvasEl.width = webcamEl.videoWidth || 640;
      canvasEl.height = webcamEl.videoHeight || 480;
    }
    frameCount++;
    requestAnimationFrame(loop);
  }
  requestAnimationFrame(loop);
}
```

Listing 5.2 — The detection loop (lines 1075-1092).

Why requestAnimationFrame instead of setInterval?

`requestAnimationFrame` pauses when the tab is backgrounded, saving CPU and battery. It also synchronises with the browser's repaint cycle, so frames are processed exactly when a new video frame is most likely to be available. Using `setInterval(loop, 33)` would keep running in the background and waste energy.

5.4 FPS Counter and Session Clock

The FPS counter is a one-second interval that copies the accumulated `frameCount` into `fps`, resets the counter, and updates the pill in the top-left of the video. The session clock is another one-second interval that computes elapsed seconds since `sessionStart` and formats them as `MM:SS` in the statusbar. Both are simple, correct, and use almost no CPU.

CHAPTER 6

Drawing the Hand Skeleton

How the 21 MediaPipe landmarks become a glowing neon skeleton on screen.

6.1 The onHandResults Entry Point

Every time MediaPipe finishes processing a frame, it calls `onHandResults(results)`. The results object contains `multiHandLandmarks` (an array of 21-point arrays, one per detected hand) and `multiHandedness` (an array of `{label: 'Left' | 'Right', score}` objects). The handler clears the canvas, updates the hand-count pill, then loops over each detected hand: drawing its skeleton and, for the primary (first) hand only, classifying the gesture, updating the confidence meter, and feeding the result into the temporal smoothing pipeline.

```
function onHandResults(results) {
  ctx.clearRect(0, 0, canvasEl.width, canvasEl.height);
  const numHands = results.multiHandLandmarks
    ? results.multiHandLandmarks.length : 0;
  handPill.textContent = `${numHands} Hand${numHands !== 1 ? 's' : ''}`;

  if (numHands === 0) {
    signOverlay.style.opacity = '0';
    updateConf(0);
    pushConfHistory(0);
    return;
  }

  for (let i = 0; i < numHands; i++) {
    const landmarks = results.multiHandLandmarks[i];
    const handedness = results.multiHandedness[i].label;
    drawLandmarks(landmarks, handedness);

    if (i === 0) {
      const { label, confidence } = classifyGesture(landmarks);
      updateConf(confidence);
      pushConfHistory(confidence);
      if (confidence >= CONFIDENCE_THRESHOLD) {
        processDetection(label, confidence);
        signOverlay.style.opacity = '1';
        olLetter.textContent = label.length > 2 ? label.slice(0,3) : label;
        olConf.textContent = `CONF ${Math.round(confidence * 100)}%`;
      } else {
        signOverlay.style.opacity = confidence > 0.35 ? '0.5' : '0';
        resetHold();
      }
    }
  }
  drawConfGraph();
}
```

Listing 6.1 — The frame handler (lines 1112-1154).

6.2 The 21 Landmarks and Their Connections

MediaPipe Hands models each hand as 21 landmarks: a wrist point (index 0), four points per finger (MCP, PIP, DIP, tip), for five fingers. The `CONNECTIONS` constant lists the 21 bone segments that should be drawn between landmarks to form the skeleton — from wrist to thumb tip, wrist to index tip, and so on, plus the palm cross-connections between finger MCPs.

6.3 Drawing Connections and Joints

The `drawLandmarks()` function draws the skeleton in two passes. First, it strokes each connection as a 2-pixel line with a coloured glow shadow (cyan for the left hand, green for the right). The x-coordinate is mirrored with $(1 - \text{lm.x}) * W$ so the drawn skeleton aligns with the mirrored video. Second, it fills each joint as a filled circle — wrist and fingertips get larger white dots, other joints get smaller coloured dots — to emphasise the points that matter most for classification.

```
function drawLandmarks(landmarks, handedness) {
  const W = canvasEl.width, H = canvasEl.height;
  const CONNECTIONS = [
    [0,1],[1,2],[2,3],[3,4],           // thumb
    [0,5],[5,6],[6,7],[7,8],           // index
    [9,10],[10,11],[11,12],             // middle
    [13,14],[14,15],[15,16],            // ring
    [0,17],[17,18],[18,19],[19,20],    // pinky
    [5,9],[9,13],[13,17]                // palm cross-connections
  ];
  const color = handedness === 'Left' ? '#00e5ff' : '#39ff6a';
  ctx.strokeStyle = color;
  ctx.lineWidth = 2;
  ctx.shadowColor = color + '55';
  ctx.shadowBlur = 8;
  for (const [a, b] of CONNECTIONS) {
    const pa = landmarks[a], pb = landmarks[b];
    ctx.beginPath();
    ctx.moveTo((1 - pa.x) * W, pa.y * H); // mirrored x
    ctx.lineTo((1 - pb.x) * W, pb.y * H);
    ctx.stroke();
  }
  // Draw joints - wrists and fingertips larger
  for (let i = 0; i < landmarks.length; i++) {
    const lm = landmarks[i];
    const x = (1 - lm.x) * W, y = lm.y * H;
    const r = i === 0 ? 5 : (i % 4 === 0 ? 4 : 2.5);
    ctx.fillStyle = i % 4 === 0 ? 'fff' : color;
    ctx.beginPath();
    ctx.arc(x, y, r, 0, Math.PI * 2);
    ctx.fill();
  }
}
```

Listing 6.2 — Drawing the skeleton (lines 1157-1204).

CHAPTER 7

Rule-Based Gesture Classifier

How geometric predicates on the landmark vector become ASL letters and words.

7.1 Normalisation

Raw MediaPipe landmark coordinates are in the range $[0, 1]$ relative to the image, which means they shift with hand position and scale with hand size. Before any classification, `classifyGesture()` normalises the landmarks by translating the wrist to the origin and scaling by the distance from wrist to middle-finger MCP (a stable proxy for hand size). After normalisation, the same gesture produces the same vector regardless of where the hand is in the frame or how close it is to the camera.

```
function classifyGesture(lm) {  
  // Translate wrist to origin, scale by palm diagonal  
  const wrist = lm[0];  
  const scale = dist(lm[0], lm[9]) || 1;  
  const N = lm.map(p => ({  
    x: (p.x - wrist.x) / scale,  
    y: (p.y - wrist.y) / scale,  
    z: (p.z - wrist.z) / scale  
  }));  
  // ...classification rules follow...
```

Listing 7.1 — Landmark normalisation (lines 1215-1224).

7.2 Finger Extension Test

The next step computes five booleans — one per finger — indicating whether each finger is extended or curled. For the four non-thumb fingers, the test is simply whether the tip's y-coordinate is above (i.e. lower numerically) the PIP joint's y, which means the finger is pointing upward. For the thumb, the test is horizontal: the tip's x must be greater than the IP joint's x. The result is a 5-bit vector `[thumb, index, middle, ring, pinky]` that captures the coarse shape of the hand.

```
function fingerExtension(N) {  
  const thumb = N[4].x > N[3].x ? 1 : 0;  
  const index = N[8].y < N[6].y ? 1 : 0;  
  const middle = N[12].y < N[10].y ? 1 : 0;  
  const ring = N[16].y < N[14].y ? 1 : 0;  
  const pinky = N[20].y < N[18].y ? 1 : 0;  
  return [thumb, index, middle, ring, pinky];  
}
```

Listing 7.2 — The 5-bit finger-extension vector (lines 1359-1368).

7.3 Letter Rules

With the extension vector in hand, the classifier walks through a series of `if` statements, each testing a combination of extension bits and a small number of geometric refinements (thumb position, finger gaps, curvature tests). The first rule that matches wins, returning a label and a hand-written confidence score. For example, A is a closed fist with the thumb beside the index finger; B is four fingers extended with the thumb across the palm; L is index up plus thumb extended (the literal L shape). For letters where the static pose is genuinely ambiguous (such as G, H, K, M, R, U, V), the rules add extra geometric tests on finger gaps and relative positions.

```
// A: fist, thumb beside index
if (!idx && !mid && !ring && !pinky && N[4].y < N[3].y)
    return { label: 'A', confidence: 0.82 };

// B: all fingers up, thumb across palm
if (idx && mid && ring && pinky && !thumbUp && N[4].x < N[5].x)
    return { label: 'B', confidence: 0.80 };

// L: index up, thumb out
if (idx && !mid && !ring && !pinky && thumbUp)
    return { label: 'L', confidence: 0.84 };

// V: peace sign - index and middle spread
if (idx && mid && !ring && !pinky && idxMidGap > 0.25)
    return { label: 'V', confidence: 0.82 };

// Y: thumb and pinky out ("hang loose")
if (!idx && !mid && !ring && pinky && thumbUp)
    return { label: 'Y', confidence: 0.82 };
```

Listing 7.3 — A representative sample of letter rules (lines 1247-1325).

7.4 Numbers and Word-Signs

After letters, the classifier tests digits 0 through 5 using simple finger-count rules (0 is the O shape, 1 is index only, 2 is the V sign, 3 is index plus middle plus thumb, 4 is four fingers up, 5 is everything open). Finally, a small number of word-signs are encoded as their most distinctive static pose — HELLO is an open hand with thumb out, for instance. If no rule matches, the function returns `{ label: '?', confidence: 0.20 }`, which the temporal smoother treats as "unknown" and discards.

Why confidence is hand-written, not model-derived

Each rule returns a fixed confidence value between 0.72 and 0.84 — chosen to be above the 0.72 acceptance threshold for unambiguous signs (L, V, Y, 5) and just at threshold for more ambiguous ones (C, F, M). This is admittedly a heuristic, but it is transparent: any misclassification can be traced to a specific rule and adjusted. A future neural classifier would replace these hand-set values with calibrated probabilities.

7.5 Geometry Helpers

Five small helper functions support the rules. `dist(a, b)` computes 3D Euclidean distance. `oShape(N)` tests whether the thumb tip is close to both the index and middle tips — the O shape. `curvatureC(N)` tests for the rough C curve. `thumbF(N)` tests the thumb-index circle of the F sign. `horizontal(a, b)` tests whether two points are roughly on the same horizontal line, used for G and H. Each helper returns a boolean and is named for the shape it tests, which keeps the rules readable.

CHAPTER 8

Temporal Smoothing & Sign Acceptance

How noisy per-frame classifications become stable, confident sign commitments.

8.1 The Problem: Jittery Classifications

A single frame's classification is unreliable. Hands move, fingers flex, lighting changes, and MediaPipe's own tracking fluctuates — so consecutive frames will often disagree about which letter is being signed, even when the user is holding a perfectly stable pose. If SignBridge accepted the first frame above threshold, the translation buffer would fill with garbage. The application therefore applies two layers of smoothing before any sign is committed: a majority-vote over a sliding window, and a hold-counter that requires the winner to remain stable for several frames.

8.2 The Sliding-Window Majority Vote

`processDetection()` maintains a `smoothWindow` array of the last six classifications. On every frame above threshold, the new label is pushed in and the oldest is shifted out. The function then tallies votes and picks the label with the highest count. The vote ratio (`votes[best] / SMOOTH_SIZE`) is also written to the stability bar, giving the user immediate visual feedback about how consistent recent detections have been.

```
const smoothWindow = [];
const SMOOTH_SIZE = 6;

function processDetection(label, confidence) {
  smoothWindow.push(label);
  if (smoothWindow.length > SMOOTH_SIZE) smoothWindow.shift();

  const votes = {};
  smoothWindow.forEach(l => { votes[l] = (votes[l] || 0) + 1; });
  const best = Object.keys(votes).sort((a,b) => votes[b]-votes[a])[0];
  const voteConf = votes[best] / SMOOTH_SIZE;
  stabFill.style.width = `${Math.round(voteConf * 100)}%`;

  if (best !== holdCandidate) {
    holdCandidate = best;
    holdCounter = 0;
    return;
  }
  holdCounter++;
  if (holdCounter >= HOLD_FRAMES) {
    const now = Date.now();
    if (best !== lastLetter || now - lastLetterTime > LETTER_DELAY_MS * 2.5) {
      window.acceptSign(best, confidence);
      lastLetter = best;
      lastLetterTime = now;
      holdCounter = 0;
    }
  }
}
```

Listing 8.1 — Sliding-window smoothing and hold counter (lines 1391-1424).

8.3 The Hold Counter

Even after the majority vote, the system does not commit immediately. The winning label must remain the winner for `HOLD_FRAMES` (18) consecutive frames. If at any point a different label wins, the counter resets to zero and the new label becomes the candidate. This ensures that only signs held deliberately — for about 0.6 seconds — get committed, filtering out transient hand positions during transitions between signs.

8.4 The Inter-Letter Delay

A second guard prevents duplicate commits. Even if a sign stays stable for many seconds, `acceptSign()` is only called when the winning label differs from `lastLetter`, or when at least `LETTER_DELAY_MS * 2.5` (1.75 seconds) have passed since the last acceptance. This means a user can hold a sign for several seconds without producing duplicate letters in the buffer, but can still spell the same letter twice in a row if they pause long enough between them.

8.5 Accepting a Sign

Once all guards pass, `window.acceptSign(best, confidence)` is called. The use of `window.` is intentional — the English translation engine monkey-patches `window.acceptSign` at the end of the script to also feed its own pipeline (see Chapter 10). Inside `acceptSign()`, word-signs (`length > 1`) are committed directly as full words; single letters are appended to `currentBuilding`. The EMERGENCY word-sign also triggers the emergency mode, and word-signs are spoken aloud if the mute flag is off.

The monkey-patch pattern

Patching `window.acceptSign` lets the English translation engine tap into the existing sign-acceptance pipeline without modifying the original function. The original is preserved as `_prevAcceptSign` and called first, then `engAcceptSign(label)` is called. This is a pragmatic way to add a second consumer to a function that was designed to have only one — it works, but a cleaner architecture would use an event emitter.

CHAPTER 9

Speech Features — TTS and STT

Closing the loop: text-to-speech for the signer, speech-to-text for the hearing user.

9.1 Text-to-Speech (TTS)

Speaking translated text aloud is handled by the Web Speech API's `SpeechSynthesis` interface. The `speakText()` helper creates a `SpeechSynthesisUtterance`, sets the language to `en-US`, the rate to 0.95 (slightly slower than default for clarity), cancels any in-progress utterance, and starts speaking. The function is a no-op if the mute flag is set, the text is empty, or the browser does not support speech synthesis.

```
function speakText(text) {  
  if (muted || !text.trim()) return;  
  if (!('speechSynthesis' in window)) return;  
  const utt = new SpeechSynthesisUtterance(text);  
  utt.lang = 'en-US';  
  utt.rate = 0.95;  
  utt.pitch = 1.0;  
  speechSynthesis.cancel();  
  speechSynthesis.speak(utt);  
}
```

Listing 9.1 — The TTS helper (lines 1558-1567).

9.2 Speech-to-Text (STT)

The reverse direction — transcribing a hearing user's speech so the signer can read it — uses `SpeechRecognition` (or its `webkitSpeechRecognition` prefix in Chromium). The `initSTT()` function checks for support, creates a recognition instance configured for continuous, interim results in `en-US`, and wires up four event handlers. `onresult` separates interim transcripts (shown in muted grey) from final ones (shown in bright text and logged to history). `onstart` and `onend` toggle the recording-pulse animation and statusbar indicator; `onerror` logs and resets.

```
recognition.onresult = e => {  
  let interim = '', final = '';  
  for (let i = e.resultIndex; i < e.results.length; i++) {  
    const t = e.results[i][0].transcript;  
    e.results[i].isFinal ? (final += t) : (interim += t);  
  }  
  sttOutput.innerHTML =  
    `<span style="color:var(--txt-hi)">${sttOutput.dataset.final || ''}</span>` +  
    `<span style="color:var(--txt-lo)">${interim}</span>`;  
  if (final) {  
    sttOutput.dataset.final = (sttOutput.dataset.final || '') + final + ' ';  
    addToHistory('[SPEECH]', final.trim());  
  }  
};
```

Listing 9.2 — The STT result handler (lines 1587-1601).

9.3 Toggling STT

`toggleSTT()` simply calls `recognition.stop()` if currently running, or `recognition.start()` if not. Before starting, the output panel and its `dataset.final` accumulator are cleared so each listening session starts fresh. The mic button in the UI is wired to this function, as is the M keyboard shortcut.

Browser support caveat

The Web Speech API's recognition half is currently only implemented in Chromium-based browsers (Chrome, Edge, Brave, Opera). Firefox and Safari users will see the mic button dimmed and the statusbar will show "STT not supported". The TTS half, by contrast, is supported everywhere. SignBridge degrades gracefully — the speech-to-text panel simply becomes read-only on unsupported browsers.

CHAPTER 10

The Offline English Translation Engine

How letter-by-letter signing becomes whole English words — without any API calls.

10.1 Design Goal: 100% Offline

The most distinctive feature of SignBridge is its second translation panel: an entirely offline English translation engine that runs in parallel with the raw letter buffer. While the Sign-to-Text panel shows the user's raw spelling (H-E-L-L-O becomes "HELLO" with no intelligence), the English panel accumulates letters, performs longest-prefix matching against a built-in dictionary of 400+ common words, and emits whole English words with their definitions. No network request is made — everything happens in JavaScript.

10.2 The Dictionary

The `EN_DICT` constant is a nested object organised by category — greetings, pronouns, verbs, common words, people, places, food, health, numbers. Each word maps to a short meaning string. The categories serve double duty: they make the dictionary human-readable, and they are surfaced in the meanings strip so the user can see at a glance whether a word is a verb, a noun, a greeting, and so on. After definition, two flat indexes are derived: `WORD_LOOKUP` (word to {meaning, category}) and `ALL_WORDS` (an array sorted by length descending, so longest-match wins).


```

const EN_DICT = {
  greetings: {
    hello: 'a greeting',
    hi: 'informal greeting',
    bye: 'farewell',
    goodbye: 'farewell',
  },
  pronouns: {
    i: 'first person singular', me: 'object form of I',
    my: 'belonging to me', you: 'second person',
    he: 'third person male', she: 'third person female',
    // ...
  },
  verbs: {
    am: 'to be (1st person)', is: 'to be (3rd person)',
    have: 'to possess', go: 'to move',
    // ... 60+ entries
  },
  // ... 6 more categories
};

const WORD_LOOKUP = {};
const ALL_WORDS = [];
for (const [cat, entries] of Object.entries(EN_DICT)) {
  for (const [word, meaning] of Object.entries(entries)) {
    WORD_LOOKUP[word] = { meaning, category: cat };
    ALL_WORDS.push(word);
  }
}
ALL_WORDS.sort((a, b) => b.length - a.length);

```

Listing 10.1 — Dictionary construction (lines 1843-2027).

10.3 Word-Sign Direct Mappings

Some signs map directly to whole words rather than letters — HELLO, THANK YOU, YES, NO, PLEASE, SORRY, HELP, WATER, FOOD, EMERGENCY. The `WORD_SIGN_MAP` constant lists these ten mappings. When one of these word-signs is accepted, the English engine commits it immediately without going through the letter-by-letter building process. This makes the engine feel intelligent: signing the word-sign for "WATER" produces the word "water" instantly, rather than requiring the user to spell W-A-T-E-R.

10.4 The Building Buffer

For letter-by-letter input, the engine maintains a `engBuilding` string. Each accepted single-letter sign is appended to it, and `engUpdateBuilding()` re-runs the matching algorithm live as letters arrive. The current best match is shown next to the building buffer with an arrow ("to water") if the input is a prefix of the match, or an approximator ("approx water") if the match is fuzzy. This gives the user real-time feedback that their spelling is being recognised as a word.

10.5 Matching Algorithm

`findBestMatch(input)` tries three strategies in order. First, an exact match in `WORD_LOOKUP`. Second, a prefix match — the longest word in `ALL_WORDS` that starts with the input. Third, a fuzzy match that scores every word by counting leading character matches and penalising length difference; if the best fuzzy score is at least 2, that word is returned. If nothing matches, the function returns `null` and the building buffer commits the raw spelling with a "spelled: XYZ" annotation.

```
function findBestMatch(input) {
  if (!input) return null;
  // 1. Exact match
  if (WORD_LOOKUP[input]) return { word: input, ...WORD_LOOKUP[input] };
  // 2. Prefix match - longest word that starts with input
  const prefixMatch = ALL_WORDS.find(w => w.startsWith(input));
  if (prefixMatch) return { word: prefixMatch, ...WORD_LOOKUP[prefixMatch] };
  // 3. Fuzzy match - most matching leading chars (with length penalty)
  let best = null, bestScore = 0;
  for (const w of ALL_WORDS) {
    let score = 0;
    for (let i = 0; i < Math.min(input.length, w.length); i++) {
      if (input[i] === w[i]) score++;
      else break;
    }
    const penalty = Math.abs(input.length - w.length) * 0.5;
    const final = score - penalty;
    if (final > bestScore) { bestScore = final; best = w; }
  }
  if (bestScore >= 2 && best) return { word: best, ...WORD_LOOKUP[best] };
  return null;
}
```

Listing 10.2 — The three-tier matcher (lines 2128-2153).

10.6 Committing a Word

When the user presses Space (or the Space button, or accepts a suggestion chip), `engCommitBuilding()` runs the matcher on the current buffer. If a match is found, the matched word is committed via `engCommitWord()`, which pushes it onto `engSentence`, records its meaning, clears the building buffer, re-renders the output, meanings, and suggestion rows, and speaks the word aloud. If no match is found, the raw lowercased spelling is committed with a "spelled: XYZ" meaning annotation, so nothing is lost.

10.7 Rendering the Sentence

`renderEngOutput()` joins the sentence array with spaces and capitalises the first letter of the first word. A blinking cursor block is appended after the last word to signal that input is still live. `renderEngMeanings()` lists each unique word in the sentence with its meaning and a small category tag. `renderEngSuggestions()` shows up to seven dictionary words that start with the current building prefix, letting the user tap a suggestion to commit it without finishing the spelling. `renderEngChips()` shows the last 30 accepted signs as coloured chips — cyan for letters, amber for word-signs — giving a compact visual history of the signing session.

CHAPTER 11

UI Wiring, Keyboard Shortcuts & Emergency Mode

Connecting buttons to behaviour, accessibility shortcuts, and the SOS safety net.

11.1 Button Wiring

Every button in the HTML has its click handler attached at the bottom of the script in a flat sequence of `addEventListener` calls. There is no event delegation, no framework, no abstraction layer — each button gets its own listener. This is verbose but extremely debuggable: to find what a button does, search for its ID. The buttons fall into four groups: camera controls (start, record), text controls (clear, undo, space, speak, copy, download), speech controls (mic, stt-copy, stt-clear), and English panel controls (eng-clear, eng-copy, eng-speak).

11.2 Keyboard Shortcuts

A single `keydown` listener on the document handles all keyboard shortcuts. It first checks that the focus is not in an input or select (so typing in form fields does not trigger shortcuts), then switches on the lowercased key. The shortcuts are: V for voice mute toggle, H for high-contrast, F for fullscreen, M for microphone, A for AI explain, Ctrl+E for emergency, Space for committing the current word, Backspace for undo, and Escape to dismiss an active emergency banner.

```
document.addEventListener('keydown', e => {
  if (e.target.tagName === 'INPUT' || e.target.tagName === 'SELECT') return;
  switch (e.key.toLowerCase()) {
    case 'v': $('mute-btn').click(); break;
    case 'h': $('hc-btn').click(); break;
    case 'f': $('fs-btn').click(); break;
    case 'm': $('mic-btn').click(); break;
    case 'a': if (!e.ctrlKey) explainSigns(currentMode); break;
    case 'e': if (e.ctrlKey) { e.preventDefault(); triggerEmergency(); } break;
    case ' ': e.preventDefault(); commitCurrentWord(); break;
    case 'backspace': e.preventDefault(); $('undo-btn').click(); break;
    case 'escape': if (emergencyActive) {
      $('emergency-banner').style.display='none';
      emergencyActive=false;
    } break;
  }
});
```

Listing 11.1 — The keyboard shortcut handler (lines 1810-1825).

11.3 Emergency Mode

The emergency feature is SignBridge's safety net. When triggered — either by signing the EMERGENCY word-sign, clicking the SOS button, or pressing Ctrl+E — the `triggerEmergency()` function shows a fixed red banner at the top of the screen, speaks "Emergency! Help is needed!" aloud, and shows a toast notification. If the browser supports geolocation, it requests the user's position and updates the banner with the latitude and longitude. The banner auto-dismisses after 30 seconds, or can be dismissed immediately with Escape.

```
function triggerEmergency() {
  if (emergencyActive) return;
  emergencyActive = true;
  $('emergency-banner').style.display = 'block';
  speakText('Emergency! Help is needed!');
  toast('Emergency mode activated');

  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(pos => {
      const { latitude, longitude } = pos.coords;
      $('emergency-banner').textContent =
        `EMERGENCY SOS - Help is needed! Location: ${latitude.toFixed(4)}, ${longitude.toFixed(4)}`;
    });
  }
  setTimeout(() => {
    $('emergency-banner').style.display = 'none';
    emergencyActive = false;
  }, 30000);
}
```

Listing 11.2 — The emergency handler (lines 1662-1683).

11.4 History Logging

Both sign acceptances and speech transcripts are logged to a shared `historyEntries` array via `addToHistory(source, text)`. Each entry stores its source tag ([SIGN] or [SPEECH]), the text, and a timestamp. The history is rendered as a scrollable list with source-coloured tags and timestamps in monospace, capped at 50 entries. This gives both users a quick way to scroll back through the conversation without re-signing or re-speaking.

11.5 Toast Notifications and Status Indicators

`toast(msg)` shows a brief notification at the bottom of the screen for 2.5 seconds — used for confirmations ("Copied to clipboard"), warnings ("Camera access denied"), and hints ("Shortcuts: V=voice, M=mic..."). The statusbar at the very bottom of the window shows four indicators: camera state (off/active), MediaPipe state (loading/ready/failed), speech state (idle/listening), and the current session time. Each indicator is a small dot plus a text label, updated by the `updateStatus()` helper.

CHAPTER 12

Summary, Strengths & Extension Points

What SignBridge does well, where it struggles, and how to take it further.

12.1 Architectural Strengths

SignBridge's most consequential architectural decision is its single-file, no-backend design. By inlining every byte of HTML, CSS and JavaScript into one `index.html`, the application becomes trivially deployable — it can be opened from a USB stick, hosted on any static server, or even emailed as an attachment. The absence of a backend eliminates server costs, latency, and privacy concerns: no video or audio ever leaves the user's machine. This makes SignBridge particularly well-suited to sensitive environments like schools, hospitals, and domestic use, where cloud-based alternatives would face procurement and compliance barriers.

A second strength is the clean separation between perception (MediaPipe), decision (the rule-based classifier), and rendering (Canvas + DOM). Each layer has a single, well-defined responsibility, and the interfaces between them are simple data structures — landmark arrays, label-plus-confidence tuples, translation-buffer strings. This separation is what makes the planned neural-classifier upgrade feasible: only `classifyGesture()` needs to change, and the rest of the pipeline can stay exactly as it is.

12.2 Known Limitations

The rule-based classifier is the most obvious limitation. It recognises a static pose for each sign, but real ASL is dynamic — many signs are distinguished by motion rather than pose (J vs. Z, for example, differ only in the tracing motion). The current classifier cannot capture this; a sequence of frames would need to be fed into a recurrent or temporal model. Additionally, the rules are tuned for one specific hand size, lighting condition, and camera angle; performance will degrade in unusual environments. The hand-written confidence values are also a weak point — they are not calibrated probabilities, and the 0.72 threshold was chosen empirically rather than derived from data.

A second limitation is browser support. The Web Speech API's recognition half is Chromium-only, so Firefox and Safari users lose the speech-to-text direction entirely. The MediaPipe Hands library requires WebAssembly, which is universally supported now but adds a ~5 MB initial download. Finally, the application has no persistence — a page refresh clears all state, including conversation history. Adding `localStorage` or `IndexedDB` persistence would be a straightforward improvement.

12.3 Extension Points

Several extension points are explicitly marked in the code. The most important is the classifier: the comment above `classifyGesture()` notes that the function can be replaced with a `tf.model.predict()` call on the 63-dimensional normalised landmark vector, and TensorFlow.js is already loaded for exactly this purpose. The `GESTURE_LABELS` array and the `WORD_SIGN_MAP` would also need to expand to cover the model's larger output vocabulary.

Extension	Where to change	Effort
Neural classifier	Replace <code>classifyGesture()</code> body	Medium — needs trained model
More sign languages	Add per-language rule sets	High — needs native-signer data
Persistence	Wrap state in <code>localStorage</code> helpers	Low — mechanical change
Mobile support	Add touch handlers, responsive CSS	Medium — UI rework
Conversation export	Add download-as-JSON button	Low — new button + handler
Multi-user sessions	Add <code>WebSocket</code> signalling layer	High — breaks no-backend promise

Table 12.1 — Practical extension points and their complexity.

12.4 Closing Thoughts

SignBridge is a useful case study in what can be built with the modern browser platform. In fewer than 2,300 lines of code, it combines real-time computer vision (MediaPipe), geometric classification (the rule engine), temporal signal processing (the smoothing pipeline), bidirectional speech I/O (Web Speech API), and a domain-specific offline translation engine (the dictionary matcher) — all without a backend, a build step, or any external dependency beyond four CDN scripts. The result is an application that is genuinely useful, instantly deployable, and small enough to read in a single sitting. For a final year project, that is exactly the right shape: ambitious enough to demonstrate real engineering judgement, restrained enough to actually ship.

End of the SignBridge Code Explanation Book. Source file: Pasted Content_1782678042968.txt (~2,262 lines, single-file HTML).